

Date	15 March 2026
Team ID	XXXXXX
Project Name	VBCUA — Voice-Based Concept Understanding Analyser
Maximum Marks	5 Marks

CODE LAYOUT, READABILITY AND REUSABILITY

CODE-LAYOUT, READABILITY AND REUSABILITY ASSESSMENT:

This document evaluates the quality of the codebase in terms of its structure, readability, and reusability. Well-organized code with proper naming conventions, comments, and modular design ensures that the project is maintainable and extensible.

CODE LAYOUT CHECKLIST

S.No	Code Quality Parameter	Description	Followed (Yes/No/Partial)	Remarks
1	Consistent Indentation	Uniform spacing/ tabs used throughout the code.	Yes	PEP 8 standard compliant.
2	Proper File Structure	Files and folders are logically organized into modular layers.	Yes	Clean separation between <code>src/</code> , <code>data/</code> , and <code>reports/</code> .
3	Meaningful Variable Names	Variables reflect their purpose clearly without ambiguity.	Yes	Descriptive tags used (e.g., <code>semantic_score</code> , <code>filler_count</code>).
4	Function/ Method Names	Functions are descriptively named using snake_case verbs.	Yes	Explaining intent clearly (e.g., <code>transcribe_audio</code> , <code>compute_overall_score</code>).
5	Code Comments	Inline and block comments explain logic where required.	Yes	Docstrings initialized on entry points and pipeline modules.
6	Modular Design	Code is split into independent	Yes	Decoupled architecture allows isolating transcription from similarity grading.

S.No	Code Quality Parameter	Description	Followed (Yes/No/Partial)	Remarks
		reusable functions or modules.		
7	No Redundant Code	Duplicate or unused code is removed to simplify maintenance.	Yes	Central configuration parameters routed via <code>config.py</code> .
8	Error Handling	Exceptions and API connection drops are handled gracefully.	Yes	Fallback engine activates smoothly if Gemini API keys are invalid.

REUSABLE COMPONENTS / MODULES

S.No	Component / Module Name	Language / Technology	Reusability Level (High / Medium / Low)
1	Database Layer (database.py)	Python / SQLAlchemy ORM	High
2	Audio Fluency Diagnostics (audio_analysis.py)	Python / Librosa & NumPy	High
3	Central Settings Routing (config.py)	Python / python-dotenv	High

OVERALL CODE QUALITY ASSESSMENT

Aspect	Assessment Description
Code Layout & Structure	Highly systematic. Code flows linearly matching execution sequences from user audio uploads to storage persistence, adhering closely to the baseline project layout blueprint.
Readability	Excellent execution. Meaningful descriptors, standard spacing formats, and structured module functions keep complex ML routing understandable for new contributors.
Reusability	Extremely adaptive. The database mapper schema, central config file, and acoustic evaluation handlers are structured to integrate seamlessly into alternative frameworks or external production cloud setups.
Documentation / Comments	Comprehensive and descriptive. Essential operational rules, configuration tables, and pipeline parameters are well annotated inside inline logs and README definitions.